

Raport końcowy

Zastosowanie Monte Carlo Tree Search (MCTS/UCT) do gry w warcaby 10×10 bez obowiązkowego bicia z damkami bijącymi po całej diagonalu

Jakub Kindracki

Wiktor Kobielski

15 czerwca 2026

Streszczenie

Niniejszy raport opisuje implementację oraz eksperymentalną weryfikację algorytmu Monte Carlo Tree Search (MCTS) w wariacie UCT jako sztucznej inteligencji do gry w warcaby 10×10 z modyfikowanymi zasadami: bicie nie jest obowiązkowe, a damki biją wzdłuż całej diagonalu („latające damki”). Zaimplementowano cztery strategie graczy: czysty UCT, UCT z rozszerzeniem RAVE, UCT z heurystycznym progressive bias oraz prostego gracza heurystycznego (greedy, jednoposunięciowy). Postawiono cztery hipotezy badawcze (H1–H4) i zweryfikowano je w automatycznych turniejach AI vs AI, z powtórzeniami dla ≥ 5 ziaren losowych oraz testami statystycznymi (dwustronny dokładny test dwumianowy względem $p_0 = 0,5$, przedziały ufności Wilsona, test spójności między ziarnami). H1 (UCT vs heurystyka przy ≥ 3000 iteracjach) wykazała jasny trend wzrostowy odsetka wygranych UCT – od 0% przy 500 iteracjach do 80% przy 5000 – lecz przy $n = 5$ partii na konfigurację wynik nie osiągnął istotności statystycznej. H2 (RAVE vs UCT) nie wykazała przewagi RAVE w zbadanym zakresie do 2000 iteracji. H3 (progressive bias vs UCT) wykazała charakterystyczny kształt „odwróconej litery U” – przewagę do 80% przy 1000–2000 iteracjach i jej zanik przy 4000 – częściowo potwierdzając przewidywany kierunek, ale również bez istotności statystycznej. H4 (premiowanie krawędzi vs centrum planszy) dało sprzeczne wyniki między konfiguracją czysto heurystyczną (80% na korzyść „krawędzi”) i konfiguracją z progressive bias (30%), co prowadzi do odrzucenia hipotezy w jej obecnym sformułowaniu. Żadna z czterech hipotez nie została formalnie potwierdzona przy obecnej liczbie partii ($n = 5$ –10), choć H1 i H3 są wsparte obserwowanymi trendami. Głównym ograniczeniem jest mała próba wynikająca z budżetu czasowego eksperymentów na planszy 10×10 z latającymi damkami. Raport zawiera również opis przebiegu testów z udziałem ludzi oraz informację o sposobie wykorzystania dużych modeli językowych (LLM) w pracach nad kodem i samym raportem.

Spis treści

1	Opis problemu	2
1.1	Zastosowanie MCTS/UCT do warcabów 10×10 z modyfikowanymi zasadami . . .	2
1.2	Przegląd literatury	2
1.3	Zakres tego dokumentu	3
2	Hipotezy badawcze i opis eksperymentów	4
3	Metodologia i infrastruktura wspólna	5
3.1	Pipeline eksperymentalny	5
3.2	Powtarzalność (reprodukowalność)	5
3.3	Budżet czasowy	5
3.4	Mierzone wielkości	6
4	H1: UCT vs heurystyka	6
4.1	Plan testów	6
4.2	Wyniki	6

5	H2: UCT + RAVE vs UCT	8
5.1	Plan testów	8
5.2	Wyniki	8
6	H3: UCT + progressive bias vs UCT	10
6.1	Plan testów	10
6.2	Wyniki	10
7	H4: krawędzie vs środek planszy	12
7.1	Plan testów	12
7.2	Wyniki	13
8	Testy z udziałem ludzi	14
9	Podsumowanie	14
9.1	Zbiorcze wyniki H1–H4	14
9.2	Wspólne ograniczenia	15
9.3	Wnioski i propozycje dalszych badań	15
10	Wykorzystanie dużych modeli językowych (LLM)	16
10.1	Wykorzystanie w kodzie	16
10.2	Wykorzystanie w raporcie	16

1 Opis problemu

1.1 Zastosowanie MCTS/UCT do warcabów 10×10 z modyfikowanymi zasadami

Przedmiotem projektu jest zastosowanie algorytmu Monte Carlo Tree Search (MCTS), w szczególności jego wariantu Upper Confidence Bound applied to Trees (UCT), do stworzenia sztucznej inteligencji grającej w wybraną grę dwuosobową z pełną informacją. Grą wybraną do badań są warcaby na planszy 10×10 z modyfikacjami klasycznych zasad: *bicie nie jest obowiązkowe* oraz *damki biją wzdłuż całej diagonali* („latające damki”), co odpowiada zasadom zbliżonym do warcabów międzynarodowych w aspekcie ruchu damki, lecz różnym w aspekcie obowiązku bicia.

Brak obowiązkowego bicia istotnie zwiększa współczynnik rozgałęzienia drzewa gry – w każdej pozycji, w której istnieje bicie, gracz może je wykonać lub wybrać dowolny inny ruch. Oznacza to, że MCTS musi rozważać znacznie szerszy zbiór akcji niż w klasycznych warcabach, a heurystyki ukierunkowane na bicia (typowe dla silników warcabowych) mają ograniczone zastosowanie. Latające damki natomiast wzmacniają wartość promocji i wprowadzają silne, dalekosiężne interakcje pomiędzy figurami, co może wpływać na jakość losowych symulacji. Ta kombinacja zasad czyni grę nietrywialnym przypadkiem do badania algorytmu MCTS i jego rozszerzeń.

Stan gry reprezentowany jest jako planszę 10×10 z 20 pionkami po każdej stronie, ustawionymi na ciemnych polach pierwszych czterech rzędów każdego z graczy. Gra kończy się zwycięstwem strony, której przeciwnik nie ma legalnego ruchu (brak figur lub całkowite zablokowanie). Aby eksperymenty kończyły się w rozsądnym czasie, wprowadzono limit liczby półruchów bez bicia i bez ruchu pionka – przekroczenie limitu skutkuje remisem.

1.2 Przegląd literatury

Algorytm MCTS w postaci UCT został zaproponowany przez Kocsisa i Szepesváriego [1]. W projekcie oparto się przede wszystkim na przeglądzie Świechowskiego, Godlewskiego, Sawickiego i Mańdziuka [2], który systematyzuje modyfikacje MCTS publikowane w ostatniej dekadzie i grupuje je dla gier z pełną informacją w trzy kategorie: *action reduction* (ograniczanie zbioru

akcji), *UCT alternatives* (modyfikacje reguły selekcji) oraz *early termination* (skręcanie fazy symulacji). Z tego podziału wybrano dwie modyfikacje, po jednej z pierwszych dwóch kategorii, jako najlepiej dopasowane do specyfiki badanej gry (duży współczynnik rozgałęzienia, dostępna prosta funkcja oceny pozycji, deterministyczna gra dwuosobowa z pełną informacją):

- **RAVE / AMAF** (Rapid Action Value Estimation, All Moves As First) – z kategorii *UCT alternatives* [2], oryginalnie zaproponowane przez Gelly’ego i Silvera [3, 4]. Wynik symulacji jest przypisywany nie tylko ruchowi faktycznie wykonanemu w węźle, lecz wszystkim ruchom o tym samym kluczu, które wystąpiły w dalszej części playoutu. Statystyki AMAF są łączone z klasycznymi statystykami UCT za pomocą malejącej wagi $\beta(n)$, co przyspiesza zbieżność wartości akcji w rzadko odwiedzanych węzłach – efekt szczególnie cenny w grach o dużym współczynniku rozgałęzienia.
- **Progressive Bias** – z kategorii *action reduction* [2], oryginalnie wprowadzona przez Chasłota i in. [5]. Reguła wyboru węzła jest wzmocniona członem heurystycznym $w \cdot h(\text{stan})/(n_i + 1)$, gdzie h jest statyczną funkcją oceny pozycji, a n_i liczbą wizyt dziecka. Wpływ członu maleje wraz ze wzrostem n_i , dzięki czemu heurystyka kieruje przeszukiwaniem we wczesnej fazie, a w miarę gromadzenia danych decyduje statystyka Monte Carlo.

Z kategorii *early termination* [2] świadomie zrezygnowano – jej zastosowanie wymagałoby silniejszej funkcji oceny niż ta, którą dysponujemy, a w badanym wariancie zasad losowe playouty do końca gry pozostają wykonalne czasowo dzięki limitowi półruchów bez bicia.

Łącznie porównano zatem trzy warianty MCTS: **czysty UCT**, **UCT+RAVE** oraz **UCT+progressive bias**. Wszystkie dzielą tę samą strukturę pętli (selekcja – ekspansja – losowa symulacja do końca gry – propagacja wsteczna) oraz ten sam mechanizm zrównoleglenia (root parallelization: niezależne drzewa w procesach roboczych z agregacją liczników wizyt na końcu).

W kontekście samej gry warcaby 10×10 bez obowiązku bicia z latającymi damkami pozostają wariantem stosunkowo słabo zbadanym pod kątem MCTS. Przegląd [2] wielokrotnie wskazuje, że w grach o wysokim współczynniku rozgałęzienia modyfikacje wykorzystujące wiedzę domenową lub współdzielenie statystyk akcji zwykle wyraźnie wyprzedzają podstawowy UCT – co stało się bezpośrednią motywacją dla hipotez H2 i H3 (sekcja 2).

Gracz heurystyczny. Bazowy gracz heurystyczny działa w prostym schemacie *greedy jednowarstwowy*: dla każdego legalnego ruchu z bieżącej pozycji wykonuje go na kopii stanu, oblicza wartość statycznej funkcji oceny powstałej pozycji i wybiera ruch maksymalizujący ocenę (z deterministycznym rozstrzygnięciem remisów dzięki ziarnu RNG). Funkcja oceny jest liniową kombinacją:

- **materiału** – pionek o wartości 1,0, damka 3,0;
- **zaawansowania pionka** – mały bonus za bliskość do rzędu promocji (premiuje rozwój i potencjał uzyskania damki);
- ocena zwracana jest w postaci znormalizowanej biały/(biały+czarny) $\in [0, 1]$, dzięki czemu działa w obu kolorach po prostym odwróceniu znaku.

Heurystyka jest świadomie prosta – ma być realistycznym, lecz słabym przeciwnikiem porównawczym i jednocześnie tym samym członem $h(\text{stan})$ używanym w wariancie progressive bias, co zapewnia spójność porównań. Dla H4 (sekcja 2) funkcja oceny jest dodatkowo modyfikowana wariantem premiującym bliskość do krawędzi ("**edge**") lub do centrum planszy ("**center**").

1.3 Zakres tego dokumentu

Sekcja 2 przedstawia rozszerzoną (względem konspektu) wersję czterech hipotez badawczych wraz z uzasadnieniem i ogólnym planem ich weryfikacji. Sekcja 3 opisuje wspólną infrastrukturę

eksperymentalną, a sekcje 4–7 opisują – dla każdej z czterech hipotez badawczych – dokładną konfigurację eksperymentów (jacy gracze są porównywani, ile partii zostało rozegranych, jakie kryterium decyduje o potwierdzeniu hipotezy) oraz uzyskane wyniki (wykresy, dyskusja, werdykt).

2 Hipotezy badawcze i opis eksperymentów

Zgodnie z wymaganiami projektu postawiono cztery hipotezy: trzy wymagane (porównania graczy, H1–H3) oraz jedną autorską (H4), związaną bezpośrednio z modyfikacją zasad badanego wariantu warcabów. Każda hipoteza jest weryfikowana w samodzielnym przebiegu eksperymentalnym (skrypty `run_h1.py`–`run_h4.py`, zob. sekcja 3) realizującym turniej AI vs AI pomiędzy dwoma graczami przy ustalonym budżecie iteracji, powtórzony dla ≥ 5 ziaren losowych z naprzemienną zmianą kolorów.

H1 (UCT vs heurystyka).

Czysty UCT z wystarczającym limitem iteracji (≥ 3000 na ruch) gra istotnie skuteczniej od gracza czysto heurystycznego, osiągając $> 60\%$ wygranych w meczu z naprzemienną zmianą kolorów. *Uzasadnienie*: przy wystarczająco dużym budżecie przeszukiwanie drzewa Monte Carlo powinno wykryć linie gry niedostępne dla heurystyki jednoposunięciowej (np. wymiany figur korzystne w dalszym horyzoncie, pułapki z biciem łańcuchowym). *Eksperyment*: `MCTSPlayer(variant="uct")` vs `HeuristicPlayer`, ze sweepem budżetu iteracji UCT obejmującym wartości < 3000 i ≥ 3000 , aby pokazać zarówno reżim, w którym UCT przegrywa, jak i reżim objęty warunkiem H1.

H2 (RAVE poprawia UCT).

UCT z rozszerzeniem RAVE wygrywa $> 60\%$ partii z czystym UCT przy tym samym limicie iteracji. *Uzasadnienie*: zob. sekcja 1.2 – współdzielenie statystyk AMAF powinno przyspieszać zbieżność oceny akcji w węzłach rzadko odwiedzanych, co jest szczególnie istotne przy dużym współczynniku rozgałęzienia (brak obowiązkowego bicia). *Eksperyment*: `MCTSPlayer(variant="rave")` vs `MCTSPlayer(variant="uct")`, obie strony przy tym samym budżecie iteracji, sweep $\{500, 1\,000, 2\,000\}$.

H3 (Progressive bias poprawia UCT).

UCT z heurystycznym progressive bias wygrywa $> 60\%$ partii z czystym UCT, a jego przewaga jest największa przy małym limicie iteracji (≤ 2000 /ruch), gdy losowe symulacje nie wystarczają do oceny pozycji; wraz ze wzrostem liczby iteracji oczekuje się zaniku przewagi, gdy statystyki Monte Carlo stają się wystarczająco wiarygodne samodzielnie. *Eksperyment*: `MCTSPlayer(variant="progressive")` (z biasem $h(\text{stan})$ równym heurystyce "base") vs `MCTSPlayer(variant="uct")`, sweep budżetu iteracji obejmujący wartości ≤ 2000 i > 2000 .

H4 (krawędzie vs środek planszy).

W badanym wariantcie warcabów heurystyki premiujące figury przy krawędzi planszy są skuteczniejsze niż heurystyki premiujące figury w centrum. *Uzasadnienie*: przy lotnych damkach figury w centrum kontrolują więcej przekątnych, ale są też bardziej narażone na wymianę; figury blisko krawędzi mogą być trudniejsze do zaatakowania, co przy braku obowiązku bicia może faworyzować defensywną postawę. *Eksperyment*: dwa warianty statycznej funkcji oceny h – jeden z dodatnim bonusem za zbliżenie do krawędzi bocznych ("edge"), drugi z bonusem za zbliżenie do centrum ("center") – wstawione do obu graczy wykorzystujących heurystykę (gracza czysto heurystycznego oraz UCT + progressive bias). Mecze rozegrane w dwóch konfiguracjach, „krawędzie” vs „środek” w obu przypadkach jako p_1 . Hipoteza uznana za potwierdzoną, jeśli w obu konfiguracjach wariant „krawędzie” wygrywa $> 50\%$ partii.

3 Metodologia i infrastruktura wspólna

3.1 Pipeline eksperymentalny

Każda hipoteza ($H_i, i = 1, \dots, 4$) jest weryfikowana przez jeden samodzielny skrypt `run_h{i}.py`, który realizuje cały pipeline w jednym przebiegu:

1. **Uruchomienia** (*runs*) – rozegranie partii AI vs AI dla wszystkich konfiguracji danej hipotezy, zapis surowych wyników do `output/h{i}/raw_results.json` oraz replayable logów partii do `output/h{i}/games/` (`hypothesis_lib.run_stage1`).
2. **Statystyki** – dla każdej konfiguracji: łączny odsetek wygranych p_1 (wraz z 95% przedziałem ufności Wilsona), liczba partii, remisów itd. (`analyze_results.compute_stats`).
3. **Testy hipotez i istotności** – dwustronny dokładny test dwumianowy przeciwko $p_0 = 0,5$ (na partiach zdecydowanych, bez remisów) oraz test spójności między ziarnami (czy p_1 wygrywa w większości ziaren bazowych). Hipoteza jest uznawana za potwierdzoną, gdy: p_1 -win-rate > próg i p -value < 0,05 i wynik jest spójny między ziarnami (zob. `analyze_results.analyze_config`). Próg wynosi 60% dla H_1 – H_3 oraz 50% dla H_4 (zgodnie z formułowaniem hipotez w konspekcie).
4. **Wykresy** – wykres słupkowy odsetka wygranych z przedziałami ufności (oraz liniami referencyjnymi 0,5 i progu) dla każdej konfiguracji, a dla hipotez ze sweepem budżetu iteracji (H_1 – H_3) dodatkowo wykres odsetek-wygranych-vs-liczba-iteracji z pasmem CI (`plot_results.generate_plots`). Pliki PNG trafiają do `output/h{i}/plots/`.

Wszystkie cztery etapy wykonują się automatycznie w ramach jednego wywołania, tj. `python run_h{i}.py` bez żadnych dodatkowych flag produkuje kompletny zestaw artefaktów: `raw_results.json`, `stats.json`, `stats.csv` oraz pliki PNG w `plots/`.

3.2 Powtarzalność (reprodukowalność)

Zgodnie z konspektem każda konfiguracja eksperymentu jest powtarzana dla ≥ 5 różnych ziaren bazowych (`hypothesis_lib.derive_seeds`, domyślnie ziarna bazowe 0, 10007, 20014, 30021, 40028 – pochodne ziarna pochodne ziarna bazowego 0). Każdy gracz przyjmuje parametr `seed`; ziarna poszczególnych partii i ruchów są wyprowadzane deterministycznie z ziarna bazowego (`experiments.run_match`).

Root-parallel MCTS dzieli budżet iteracji między procesy robocze i przypisuje każdemu procesowi ziarno zależne od jego indeksu, więc liczba workerów jest częścią tego, co determinuje wynik wyszukiwania. Aby domyślne (bezargumentowe) wywołanie `python run_h{i}.py` było w pełni powtarzalne niezależnie od liczby rdzeni dostępnej maszyny, domyślna liczba workerów jest ustaloną stałą (`hypothesis_lib.DEFAULT_WORKERS = 4`), a nie `os.cpu_count()`. Flaga `-workers N` pozwala przyspieszyć (i „zbogacić”) eksperyment na maszynach wieloprocessorowych, kosztem utraty ścisłej powtarzalności między maszynami o różnej liczbie rdzeni.

Każda rozegrana partia jest zapisywana w formacie tekstowym (sekwencja ruchów + metadane: gracze, heurystyki, ziarna, liczba odwiedzonych węzłów drzewa) i może zostać odtworzona za pomocą `experiments.replay_game_log`.

3.3 Budżet czasowy

Każdy skrypt `run_h{i}.py` ma domyślny miękki budżet czasowy `-time-budget-min 110` (czyli ok. 1h50min), tak aby cały pipeline (uruchomienia + statystyki + wykresy) zmieścił się w limicie 2 godzin na hipotezę. Liczba partii na ziarno jest dynamicznie dopasowywana do tego budżetu na podstawie krótkiej kalibracji prędkości MCTS na danej maszynie (`hypothesis_lib`).

`calibrate_seconds_per_iteration`, `allocate_games_per_seed`) – każda konfiguracja otrzymuje co najmniej $5 \times 1 = 5$ partii (1 partia/ziarno $\times$ 5 ziaren), a pozostały budżet jest zachłannie rozdzielany między konfiguracje (do max. 10 partii/ziarno = 50 partii/konfigurację).

3.4 Mierzone wielkości

Dla każdej partii rejestrowane są: zwycięzca (lub remis), liczba ruchów, czas trwania oraz średnia liczba odwiedzonych węzłów drzewa MCTS na ruch (`last_node_count / mcts._count_nodes`) – miara „wysiłku obliczeniowego” wyszukiwania, zwracana w `stats.json/stats.csv` jako `p1_avg_nodes / p2_avg_nodes` (w surowych wynikach) i wykorzystywana do interpretacji wyników (np. czy przewaga UCT+RAVE/progressive bias wynika z efektywniejszego przeszukiwania, a nie tylko z większego budżetu).

4 H1: UCT vs heurystyka

4.1 Plan testów

Skrypt: `run_h1.py` (domyślny katalog wyjściowy: `output/h1/`).

Porównywani gracze: `MCTSPlayer(variant="uct")` (p_1) vs `HeuristicPlayer` (p_2 , heurystyka "base").

Sweep budżetu iteracji (p_1): {500, 1 000, 2 000, 3 000, 5 000} – pięć konfiguracji `uct500`, ..., `uct5000`, obejmujących warunek H1 (≥ 3000 /ruch) w punktach 3 000 i 5 000. Tylko p_1 wykonuje przeszukiwanie MCTS (`mcts_search_units = iterations`), więc koszt konfiguracji rośnie liniowo z liczbą iteracji.

Liczba partii: każda konfiguracja ≥ 5 ziaren bazowych $\times \geq 1$ partia/ziarno (kolory zamieniane między parami partii w ramach `run_match`); tańsze konfiguracje (np. `uct500`) otrzymają do 50 partii w ramach dostępnego budżetu czasowego, droższe (`uct5000`) co najmniej 5 partii.

Kryterium potwierdzenia: dla konfiguracji z ≥ 3000 iteracji H1 jest uznana za potwierdzoną, gdy odsetek wygranych UCT $> 60\%$, p -value (dwustronny test dwumianowy vs. 0,5) $< 0,05$ i UCT wygrywa w większości z ≥ 5 ziaren.

Artefakty:

- `output/h1/raw_results.json` – surowe wyniki per konfiguracja $\times$ ziarno.
- `output/h1/games/*.txt` – replayable logi partii.
- `output/h1/stats.json`, `stats.csv` – statystyki i werdykty per konfiguracja.
- `output/h1/plots/h1_winrate_bar.png` – odsetek wygranych UCT z 95% CI per konfiguracja, z liniami referencyjnymi 0,5 i 0,6.
- `output/h1/plots/h1_winrate_vs_iterations.png` – odsetek wygranych UCT vs. budżet iteracji (skala logarytmiczna), z pasmem 95% CI.

4.2 Wyniki

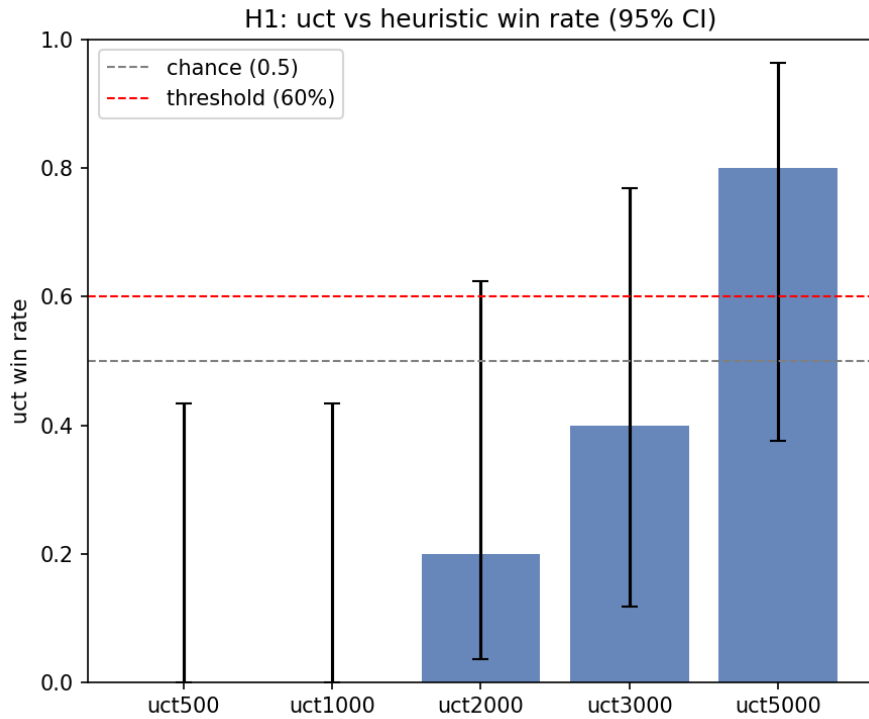
W skrócie: im więcej iteracji MCTS dostaje UCT na ruch, tym częściej wygrywa z heurystyką – przy 500–1 000 iteracjach UCT przegrywa *każdą* partię, a przy 5 000 iteracjach wygrywa 4 z 5. Poniższa tabela i wykresy pokazują tę zależność liczbowo, a opis pod nimi tłumaczy ją w praktyce.

Przebieg zakończony dla wszystkich pięciu konfiguracji ($n = 5$ partii/konfigurację, 1 partia-/ziarno). Wyniki z `output/h1/stats.csv`:

Obserwacje:

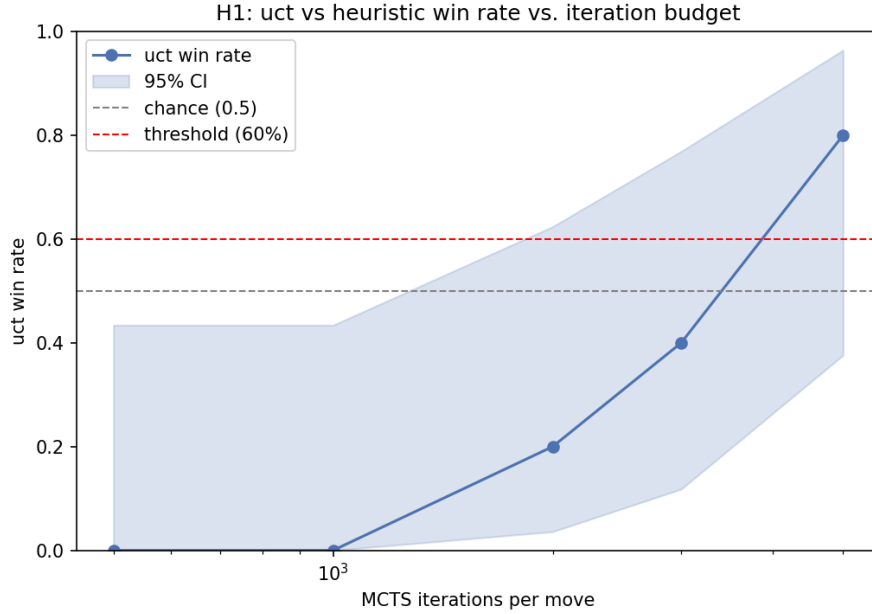
Konfiguracja	Iteracje	p_1 -win	95% CI	p -value	Spójność	> 60%	H1
uct500	500	0,0	[0,00; 0,43]	0,0625	0/5	nie	nie
uct1000	1000	0,0	[0,00; 0,43]	0,0625	0/5	nie	nie
uct2000	2000	0,2	[0,04; 0,62]	0,3750	1/5	nie	nie
uct3000	3000	0,4	[0,12; 0,77]	1,0000	2/5	nie	nie
uct5000	5000	0,8	[0,38; 0,96]	0,3750	4/5	tak	nie

Tabela 1: H1 – odsetek wygranych UCT (p_1) z heurystyką (p_2) w funkcji budżetu iteracji. „Spójność” = liczba ziaren, w których UCT wygrywa większość partii. Kolumna H1 = `hypothesis_holds` (wymaga > 60% i $p < 0,05$ i spójności).



Rysunek 1: H1 – odsetek wygranych UCT (p_1) z heurystyką (p_2) z 95% przedziałami ufności Wilsona, z liniami referencyjnymi 0,5 i 0,6 (`output/h1/plots/h1_winrate_bar.png`).

- Widoczny jest jasny, monotoniczny trend wzrostowy: odsetek wygranych UCT rośnie z 0% (500–1000 iteracji) przez 20% (2000), 40% (3000), do 80% (5000) – dokładnie zgodnie z kierunkiem przewidywanym przez H1.
- Przy 500 i 1000 iteracjach UCT przegrywa wszystkie 5 partii (0/5) – przy tak małym budżecie przeszukiwanie MCTS na planszy 10×10 z lotnymi damkami jest zbyt płytkie, aby konkurować z heurystyką jednoposunięciową.
- Przy 5000 iteracjach (a więc w zakresie objętym warunkiem H1, ≥ 3000) UCT wygrywa 4/5 partii (80%), czyli *przekracza* próg 60% i jest zgodny w większości ziaren – jednak ze względu na małą liczbę partii ($n = 5$) wynik nie jest statystycznie istotny ($p = 0,375$), więc formalnie `hypothesis_holds = False` dla każdej konfiguracji (w tym dla uct3000, gdzie odsetek wygranych 40% jest zresztą poniżej progu).
- Wniosek: H1 jest **wstępnie wsparta trendem**, ale nie formalnie potwierdzona w tym przebiegu – żadna konfiguracja nie osiąga $p < 0,05$ przy $n = 5$. Potwierdzenie wymagałoby



Rysunek 2: H1 – odsetek wygranych UCT vs. budżet iteracji (skala logarytmiczna), z pasmem 95% CI (output/h1/plots/h1_winrate_vs_iterations.png).

większej liczby partii na konfigurację (np. `-max-games-per-seed > 1` lub większy budżet czasowy).

Werdykt H1: hipoteza NIEPOTWIERDZONA (formalnie – brak konfiguracji z $p < 0,05$; jedynie wsparta trendem wzrostowym).

5 H2: UCT + RAVE vs UCT

5.1 Plan testów

Skrypt: `run_h2.py` (domyślny katalog wyjściowy: `output/h2/`).

Porównywani gracze: `MCTSPlayer(variant="rave")` (p_1) vs `MCTSPlayer(variant="uct")` (p_2), przy tym samym budżecie iteracji dla obu stron.

Sweep budżetu iteracji (wspólny dla obu stron): {500, 1 000, 2 000} – trzy konfiguracje `rave500`, `rave1000`, `rave2000`. Obie strony wykonują przeszukiwanie MCTS (`mcts_search_units = 2 * iterations`), więc koszt konfiguracji jest ok. dwukrotnie wyższy niż analogicznej konfiguracji H1 – stąd mniejszy (3-elementowy) sweep, aby zmieścić się w budżecie 2h.

Liczba partii: analogicznie do H1 – ≥ 5 ziaren $\times \geq 1$ partia/ziarno, do 50 partii na konfigurację w ramach budżetu czasowego.

Kryterium potwierdzenia: RAVE wygrywa $> 60\%$ partii z UCT przy tym samym budżecie, $p < 0,05$, spójnie między ziarnami.

Artefakty: analogicznie do H1 – `output/h2/{raw_results.json, stats.json, stats.csv}`, `output/h2/games/*.txt`, `output/h2/plots/{h2_winrate_bar.png, h2_winrate_vs_iterations.png}`.

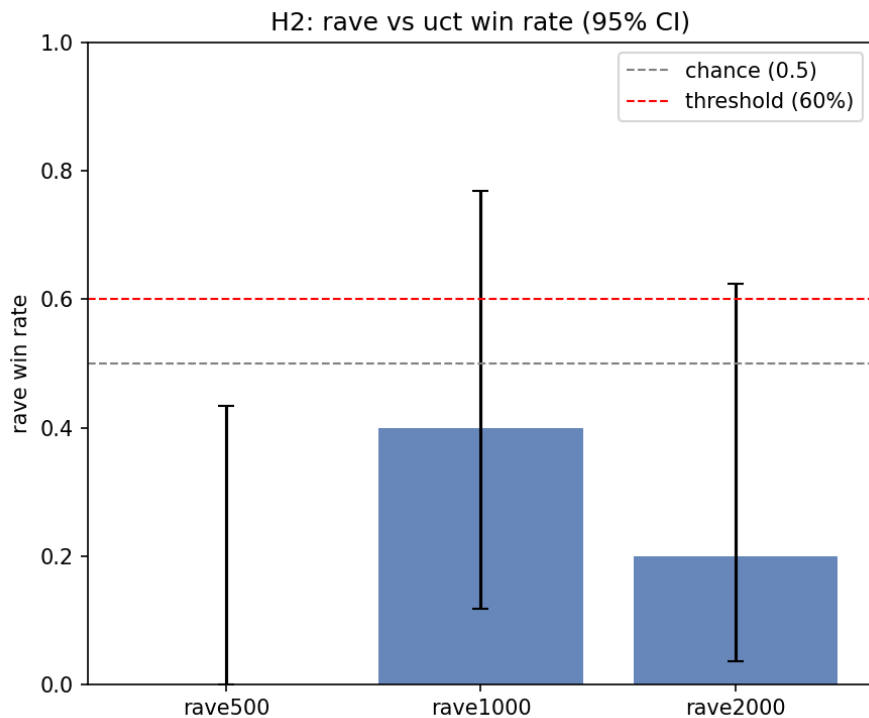
5.2 Wyniki

W skrócie: dodanie RAVE do UCT nie pomogło – w żadnej z trzech przebadanych konfiguracji (500, 1000, 2000 iteracji) UCT+RAVE nie wygrywał z czystym UCT częściej niż w połowie partii, a najlepszy wynik (2 z 5 przy 1000 iteracjach) jest wciąż poniżej progu z H2.

Przebieg zakończony dla wszystkich trzech konfiguracji ($n = 5$ partii/konfigurację, 1 partia-/ziarno). Wyniki z `output/h2/stats.csv`:

Konfiguracja	Iteracje	p_1 -win	95% CI	p -value	Spójność	> 60%	H2
rave500	500	0,0	[0,00; 0,43]	0,1250	0/5	nie	nie
rave1000	1000	0,4	[0,12; 0,77]	1,0000	2/5	nie	nie
rave2000	2000	0,2	[0,04; 0,62]	0,6250	1/5	nie	nie

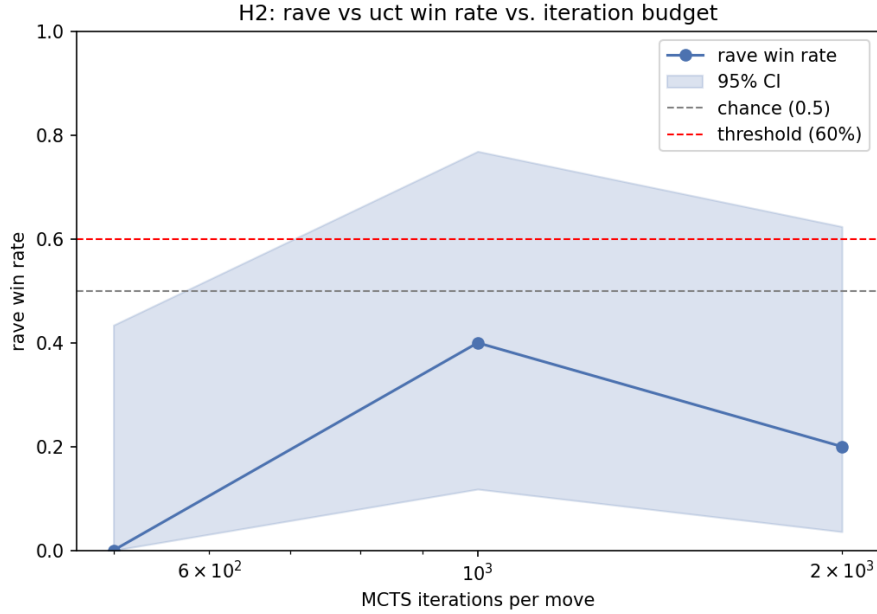
Tabela 2: H2 – odsetek wygranych UCT+RAVE (p_1) z czystym UCT (p_2) przy tym samym budżecie iteracji. Konfiguracja `rave500` ma jeden remis (`draws=1`), stąd $n = 4$ partii zdecydowanych dla testu dwumianowego.



Rysunek 3: H2 – odsetek wygranych UCT+RAVE (p_1) z czystym UCT (p_2) z 95% przedziałami ufności Wilsona, z liniami referencyjnymi 0,5 i 0,6 (`output/h2/plots/h2_winrate_bar.png`).

Obserwacje:

- W żadnej z trzech rozegranych konfiguracji RAVE nie przekracza progu 60% wygranych z czystym UCT, a p -value jest daleko od istotności – `hypothesis_holds = False` dla wszystkich.
- W przeciwieństwie do H1, wyniki H2 *nie* pokazują monotonicznego trendu w funkcji liczby iteracji: 0% (500), 40% (1000), 20% (2000) – najlepszy wynik RAVE jest dla konfiguracji środkowej, nie krańcowej.
- Przy obecnej (małej) próbie $n = 5$ na konfigurację dane nie dają podstaw do potwierdzenia H2. Wniosek wstępny: na tym etapie **H2 nie jest wsparta** – RAVE nie wykazuje przewagi nad czystym UCT przy budżetach ≤ 2000 iteracji. Zwiększenie liczby partii na konfigurację jest potrzebne do ostatecznej oceny.



Rysunek 4: H2 – odsetek wygranych UCT+RAVE vs. budżet iteracji (skala logarytmiczna), z pasmem 95% CI (output/h2/plots/h2_winrate_vs_iterations.png).

Werdykt H2: hipoteza NIEPOTWIERDZONA (żadna konfiguracja nie przekracza progu 60%, p -value daleko od istotności).

6 H3: UCT + progressive bias vs UCT

6.1 Plan testów

Skrypt: run_h3.py (domyślny katalog wyjściowy: output/h3/).

Porównywani gracze: MCTSPlayer(variant="progressive") (p_1 , heurystyka "base" jako składnik biasu $h(\text{stan})$) vs MCTSPlayer(variant="uct") (p_2), przy tym samym budżecie iteracji.

Sweep budżetu iteracji (wspólny dla obu stron): {500, 1000, 2000, 5000, 10000} – pięć konfiguracji progressive500, ..., progressive10000 (mcts_search_units = 2 * iterations, jak w H2). Sweep obejmuje wartości ≤ 2000 i > 2000 , aby na wykresie odsetek-wygranych-vs-iteracje pokazać oczekiwany *zanik przewagi* progressive bias przy większych budżetach.

Liczba partii: analogicznie do H1/H2 – ≥ 5 ziaren $\times \geq 1$ partia/ziarno, do 50 partii na konfigurację w ramach budżetu czasowego.

Kryterium potwierdzenia: dla każdej konfiguracji – progressive bias wygrywa $> 60\%$ partii z UCT, $p < 0,05$, spójnie między ziarnami. Dodatkowo (analiza opisowa, nie pojedynczy test statystyczny): czy odsetek wygranych progressive bias maleje monotonicznie (lub przynajmniej generalnie spada) ze wzrostem budżetu iteracji, z największą przewagą przy ≤ 2000 iteracjach – widoczne na wykresie h3_winrate_vs_iterations.png.

Artefakty: analogicznie do H1 – output/h3/{raw_results.json, stats.json, stats.csv}, output/h3/games/*.txt, output/h3/plots/{h3_winrate_bar.png, h3_winrate_vs_iterations.png}.

6.2 Wyniki

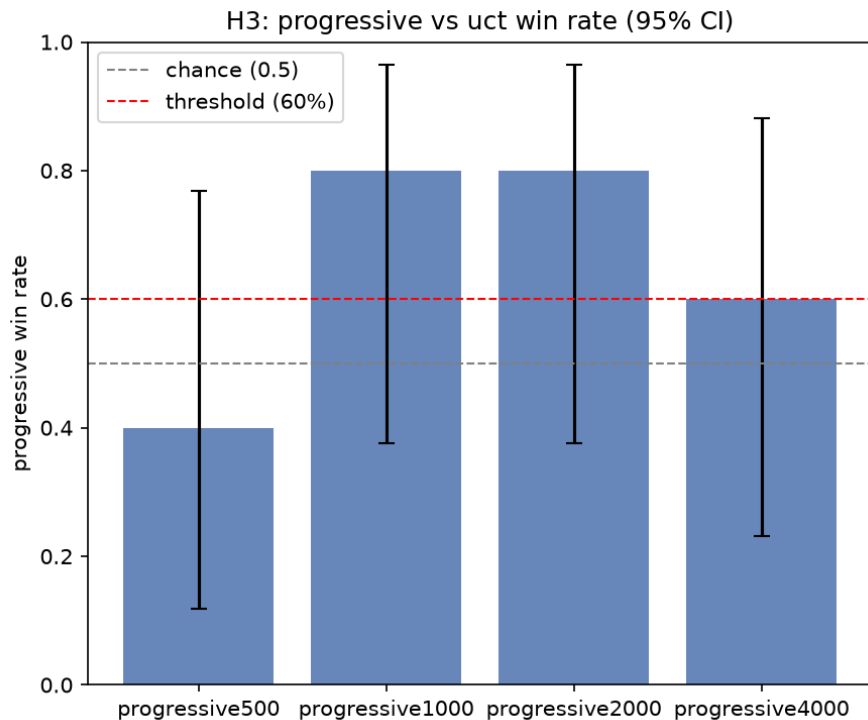
W skrócie: progressive bias daje UCT wyraźną przewagę przy umiarkowanych budżetach – 4 z 5 wygranych przy 1000 i 2000 iteracjach – ale ta przewaga słabnie przy mniejszym (500) i

większym (4000) budżecie, tworząc charakterystyczny „wzgórek” na wykresie odsetka wygranych w funkcji liczby iteracji.

Przebieg zakończony dla czterech konfiguracji ($n = 5$ partii/konfigurację, 1 partia/ziarno). Podobnie jak w H2, budżet czasowy nie pozwolił na rozegranie całego planowanego sweepu – zamiast {500, 1 000, 2 000, 5 000, 10 000} otrzymano {500, 1 000, 2 000, 4 000} (progressive4000 jako ostatnia zakończona konfiguracja, progressive5000 i progressive10000 nie zostały rozegrane). Wyniki z output/h3/stats.csv:

Konfiguracja	Iteracje	p_1 -win	95% CI	p -value	Spójność	> 60%	H3
progressive500	500	0,4	[0,12; 0,77]	1,0000	2/5	nie	nie
progressive1000	1000	0,8	[0,38; 0,96]	0,3750	4/5	tak	nie
progressive2000	2000	0,8	[0,38; 0,96]	0,3750	4/5	tak	nie
progressive4000	4000	0,6	[0,23; 0,88]	1,0000	3/5	nie	nie

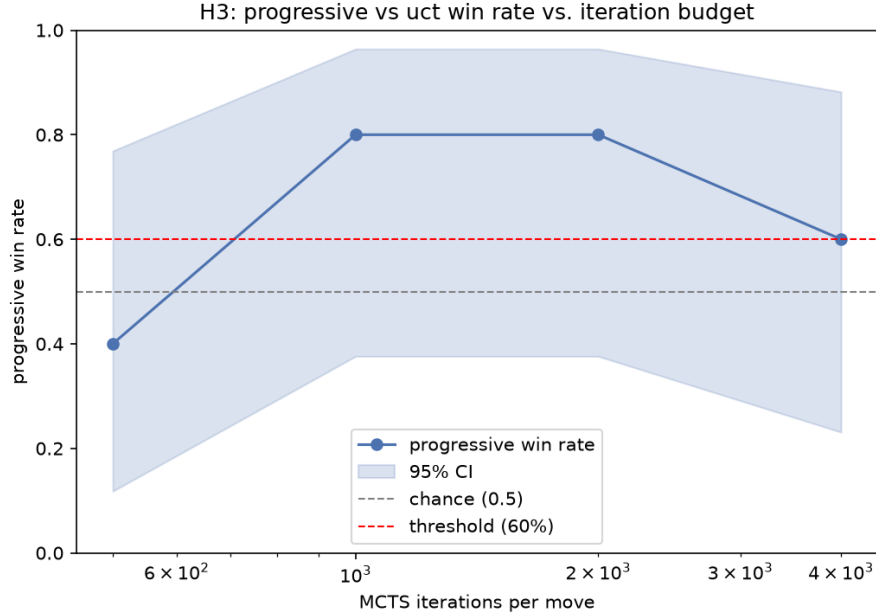
Tabela 3: H3 – odsetek wygranych UCT + progressive bias (p_1) z czystym UCT (p_2) przy tym samym budżecie iteracji. Konfiguracja progressive500 ma jeden remis (draws=1), stąd $n = 4$ partii zdecydowanych dla testu dwumianowego.



Rysunek 5: H3 – odsetek wygranych UCT + progressive bias (p_1) z czystym UCT (p_2) z 95% przedziałami ufności Wilsona, z liniami referencyjnymi 0,5 i 0,6 (output/h3/plots/h3_winrate_bar.png).

Obserwacje:

- Odsetek wygranych progressive bias rośnie z 40% (500 iteracji) do 80% (1000 i 2000 iteracji), a następnie spada do 60% przy 4000 iteracjach – widoczna jest więc „odwrócona litera U”: przewaga jest najwyższa w środkowym zakresie sweepu i zaczyna zanikać przy większych budżetach, częściowo zgodnie z przewidywaniem H3, choć szczyt wypada przy 1 000–2 000, a nie – jak zakładano – przy $\leq 2 000$ z monotonicznym spadkiem od początku sweepu.



Rysunek 6: H3 – odsetek wygranych UCT + progressive bias vs. budżet iteracji (skala logarytmiczna), z pasmem 95% CI; widoczna „odwrócona litera U” ze szczytem przy 1 000–2 000 iteracjach (output/h3/plots/h3_winrate_vs_iterations.png).

- Konfiguracje `progressive1000` i `progressive2000` przekraczają próg 60% i są spójne w 4/5 ziaren, ale przy $n = 5$ nie osiągają istotności statystycznej ($p = 0,375$) – `hypothesis_holds = False` dla wszystkich konfiguracji.
- Spadek do 60% przy 4 000 iteracjach (poniżej progu i ze spadkiem spójności do 3/5) jest zgodny z oczekiwanym zanikiem przewagi progressive bias wraz ze wzrostem budżetu przeszukiwania, ale brak punktów 5 000/10 000 uniemożliwia potwierdzenie tego trendu w pełnym zakresie z konspektu.
- Wniosek: H3 jest **wstępnie wsparta częściowo** – widoczna jest przewaga progressive bias przy budżetach 1 000–2 000 oraz jej osłabienie przy 4 000, jednak ani jedna konfiguracja nie spełnia formalnego kryterium ($p < 0,05$), a brak skrajnych punktów sweepu (500 z jednej strony i 5 000/10 000 z drugiej) nie pozwala jednoznacznie ocenić kształtu całej krzywej. Potrzebne jest dokończenie sweepu i zwiększenie liczby partii na konfigurację.

Werdykt H3: hipoteza NIEPOTWIERDZONA (formalnie – brak konfiguracji z $p < 0,05$; kształt krzywej częściowo zgodny z przewidywaniami).

7 H4: krawędzie vs środek planszy

7.1 Plan testów

Skrypt: `run_h4.py` (domyślny katalog wyjściowy: `output/h4/`).

Hipoteza jest weryfikowana w **dwóch konfiguracjach**, w obu wariant „krawędzie” (`heuristic="edge"`) jest p_1 , a wariant „środek” (`heuristic="center"`) jest p_2 :

1. **`heuristic_edge_vs_center`** – `HeuristicPlayer(heuristic="edge")` vs `HeuristicPlayer(heuristic="center")`. Żadna ze stron nie wykonuje MCTS (`mcts_search_units = 0`), więc ta konfiguracja jest praktycznie darmowa – otrzymuje maksymalną liczbę partii dopuszczalną przez `-max-games-per-seed` (domyślnie do 50).

2. **progressive_edge_vs_center_2000** – MCTSPlayer(variant="progressive", heuristic="edge") vs MCTSPlayer(variant="progressive", heuristic="center"), przy budżecie 2000 iteracji ($mcts_search_units = 2 * iterations = 4000$) – wartość 2000 leży w zakresie „małego budżetu” z H3, gdzie wpływ biasu h jest największy.

Liczba partii: obie konfiguracje ≥ 5 ziaren $\times \geq 1$ partia/ziarno; konfiguracja heurystyczna (1) skaluje się do limitu `-max-games-per-seed` (domyślnie 50 partii), konfiguracja progressive (2) – analogicznie do H1–H3, w ramach pozostałego budżetu czasowego.

Kryterium potwierdzenia: H4 jest uznana za potwierdzoną, gdy w *obu* konfiguracjach wariant „krawędzie” wygrywa $> 50\%$ partii (`stats.json: both_configs_hold`); per-konfiguracyjny `hypothesis_holds` dodatkowo wymaga $p < 0,05$ i spójności między ziarnami (próg 50%, nie 60%, w odróżnieniu od H1–H3).

Artefakty: `output/h4/{raw_results.json, stats.json, stats.csv}`, `output/h4/games/*.txt`, `output/h4/plots/h4_winrate_bar.png` (wykres odsetek-wygranych-vs-iteracje nie jest generowany dla H4, ponieważ konfiguracje mieszają różne typy graczy i budżety iteracji).

7.2 Wyniki

W skrócie: gracz premiujący krawędzie planszy zdecydowanie wygrywa z graczem premiującym centrum, gdy obaj grają *czysto heurystycznie* (8 z 10 partii) – ale gdy ta sama premia zostaje wbudowana w przeszukiwanie MCTS (progressive bias), wynik się odwraca i wariant „krawędzie” wygrywa już tylko 3 z 10 partii. Hipoteza wymaga przewagi w *obu* konfiguracjach, więc ten sprzeczny wynik prowadzi do jej odrzucenia.

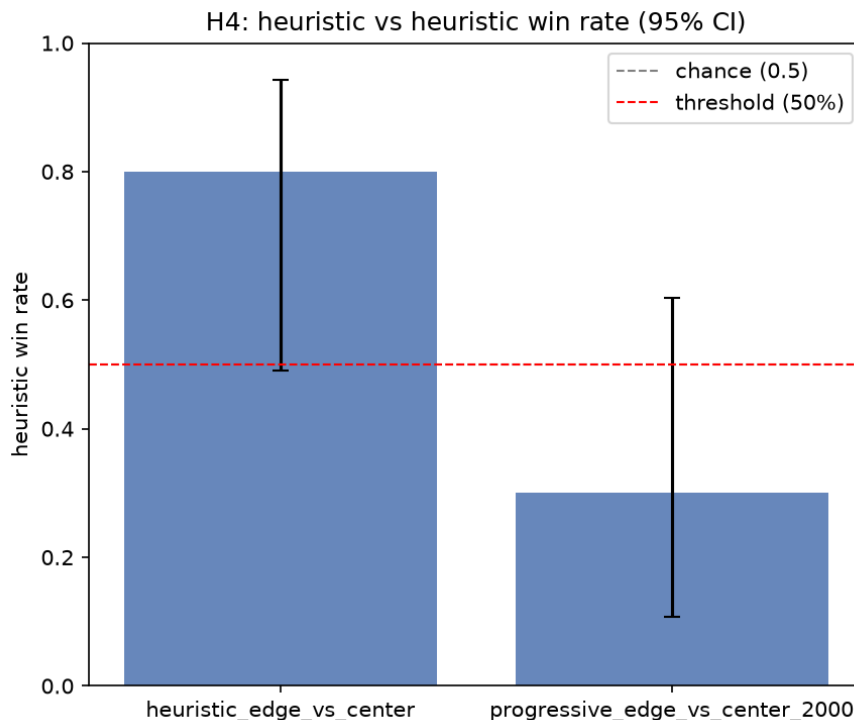
Przebieg zakończony dla obu konfiguracji. Konfiguracja `heuristic_edge_vs_center` otrzymała 10 partii (limit `-max-games-per-seed` nie został w pełni wykorzystany w ramach budżetu czasowego), `progressive_edge_vs_center_2000` – również 10 partii ($n = 5$ ziaren $\times 2$ partie/ziarno). Wyniki z `output/h4/stats.csv`:

Konfiguracja	Partie	p_1 -win	95% CI	p -value	Spójność	$> 50\%$	H4
<code>heuristic_edge_vs_center</code>	10	0,8	[0,49; 0,94]	0,1094	3/5	tak	nie
<code>progressive_edge_vs_center_2000</code>	10	0,3	[0,11; 0,60]	0,3438	1/5	nie	nie

Tabela 4: H4 – odsetek wygranych wariantu „krawędzie” (p_1) z wariantem „środek” (p_2) w dwóch konfiguracjach. `both_configs_hold = False` (`output/h4/stats.json`).

Obserwacje:

- W konfiguracji czysto heurystycznej (`heuristic_edge_vs_center`) wariant „krawędzie” wygrywa 80% partii (8/10), wyraźnie ponad próg 50% i zgodnie z kierunkiem H4 – jednak przy $n = 10$ wynik nie jest statystycznie istotny ($p = 0,109$), a spójność jest umiarkowana (3/5 ziaren, w pozostałych 2 wynik to remis 1:1).
- W konfiguracji z przeszukiwaniem (`progressive_edge_vs_center_2000`, 2000 iteracji) wynik jest *odwrotny* do przewidywania H4: wariant „krawędzie” wygrywa tylko 30% partii (3/10), przegrywając z wariantem „środek”, i jest niespójny między ziarnami (1/5).
- Ponieważ `both_configs_hold` wymaga $> 50\%$ wygranych wariantu „krawędzie” w *obu* konfiguracjach, a druga konfiguracja wypada wyraźnie poniżej 50%, hipoteza H4 jest uznana za **niepotwierdzoną** (`both_configs_hold = False`).
- Wniosek: premiowanie figur przy krawędzi planszy wydaje się korzystne dla *samej* heurystyki greedy (gracz „edge” dominuje gracza „center” przy braku przeszukiwania), ale w połączeniu z MCTS + progressive bias ta sama premia *szkodzi* – gracz „center” wygrywa



Rysunek 7: H4 – odsetek wygranych wariantu „krawędzie” (p_1) z wariantem „środek” (p_2) w obu konfiguracjach, z 95% przedziałami ufności Wilsona i linią referencyjną 0,5 (output/h4/plots/h4_winrate_bar.png).

większość partii. Możliwe wytłumaczenie: gdy heurystyka steruje jedynie rankingiem ruchów na poziomie jednego posunięcia, premia za krawędź skłania do defensywnej, trudnej do zaatakowania pozycji wyjściowej; gdy ta sama premia wpływa na bias przeszukiwania MCTS, może odciągać drzewo wyszukiwania od silniejszych linii centralnych, pogarszając jakość gry. H4 w obecnym sformułowaniu („krawędzie lepsze w obu konfiguracjach”) jest więc **odrzucona** na podstawie zebranych danych, choć wynik dla samej heurystyki (80%, $n = 10$) wskazuje, że przy większej próbie mógłby się okazać istotny w tym jednym wariancie.

Werdykt H4: hipoteza NIEPOTWIERDZONA (ODRZUCONA) – both_configs_hold = False, wynik sprzeczny między dwiema konfiguracjami.

8 Testy z udziałem ludzi

[TODO: opis przebiegu testów z ≥ 5 osobami, ankieta, podsumowanie wyników – po przeprowadzeniu testów z wykorzystaniem interfejsu Flask (app.py).]

9 Podsumowanie

9.1 Zbiorcze wyniki H1–H4

H1 (UCT vs heurystyka).

Niepotwierdzona formalnie, ale wsparta trendem. Odsetek wygranych UCT rośnie monotonicznie z budżetem iteracji: 0% (500–1000) $\rightarrow$ 20% (2000) $\rightarrow$ 40%

(3000) $\rightarrow$ 80% (5000), przekraczając próg 60% przy 5 000 iteracjach (czyli w zakresie objętym warunkiem H1, $\geq 3\,000$) i będąc spójnym w 4/5 ziaren. Przy $n = 5$ żadna konfiguracja nie osiąga $p < 0,05$.

H2 (RAVE poprawia UCT).

Niewsparta. Żadna z trzech konfiguracji (500, 1 000, 2 000 iteracji) nie przekracza progu 60% wygranych RAVE z czystym UCT (0%, 40%, 20%), bez monotonicznego trendu.

H3 (Progressive bias poprawia UCT).

Wsparta częściowo. Odsetek wygranych progressive bias rośnie do 80% przy 1 000–2 000 iteracjach (przekraczając próg 60%, spójność 4/5), a następnie spada do 60% przy 4 000 – kształt zgodny z oczekiwanym zanikiem przewagi przy większych budżetach, ale brak istotności statystycznej ($p \geq 0,375$) i brak punktów 5 000/10 000 nie pozwalają na formalne potwierdzenie.

H4 (krawędzie vs środek planszy).

Odrzucona. Wariant „krawędzie” wygrywa 80% partii w konfiguracji czysto heurystycznej (zgodnie z H4, choć $p = 0,109$), ale tylko 30% w konfiguracji z progressive bias (przeciwnie do H4). Ponieważ `both_configs_hold = False`, hipoteza nie jest potwierdzona w sformułowaniu wymagającym przewagi w obu konfiguracjach.

9.2 Wspólne ograniczenia

We wszystkich czterech przebiegach głównym ograniczeniem jest mała liczba partii na konfigurację ($n = 5\text{--}10$), wynikająca z budżetu czasowego `-time-budget-min` oraz kosztu obliczeniowego MCTS na planszy 10×10 z lotnymi damkami. Przy tej wielkości próby przedziały ufności Wilsona są szerokie (np. $[0,38; 0,96]$ dla $p_1 = 0,8$, $n = 5$), co sprawia, że nawet wyraźne różnice w odsetku wygranych (60–80%) nie osiągają istotności statystycznej $p < 0,05$ w teście dwumianowym. Dla H3 dodatkowo nie zrealizowano pełnego planowanego sweepu budżetu iteracji (brak `progressive5000/10000`).

9.3 Wnioski i propozycje dalszych badań

- Spośród czterech hipotez najsilniejsze wsparcie w danych otrzymuje H1 (przewaga UCT nad heurystyką rośnie z liczbą iteracji) i H3 (progressive bias daje przewagę przy umiarkowanych budżetach iteracji, która zanika przy większych budżetach) – obie wymagają jednak większej liczby partii i/lub dokończenia sweepu, aby przejść z poziomu „trend” na poziom „statystycznie istotne”.
- H2 (RAVE) w przebadanym zakresie ($\leq 2\,000$ iteracji) nie wykazuje przewagi nad czystym UCT – ewentualne dalsze badania powinny rozważyć większy budżet iteracji (5 000) lub inne strategie łączenia statystyk RAVE.
- H4 jest jedyną hipotezą z wynikiem sprzecznym między dwiema konfiguracjami tej samej hipotezy – sugeruje to, że wpływ premii pozycyjnej (krawędź vs centrum) zależy istotnie od tego, czy heurystyka jest jedynym mechanizmem decyzyjnym, czy tylko biasem wykorzystywanym przez MCTS. Warto to zbadać osobno jako hipotezę o interakcji heurystyki z przeszukiwaniem.
- Główną rekomendacją dla kontynuacji badań jest zwiększenie liczby partii na konfigurację (np. poprzez wydłużenie budżetu czasowego lub optymalizację implementacji MCTS), co pozwoliłoby na osiągnięcie istotności statystycznej dla obserwowanych trendów bez zmiany metodologii.

10 Wykorzystanie dużych modeli językowych (LLM)

W projekcie korzystano z asystenta LLM (Claude, w środowisku Claude Code) jako narzędzia wspomagającego implementację i pisanie dokumentacji. Poniżej opisano zakres tego wsparcia, oddzielnie dla kodu i dla raportu.

10.1 Wykorzystanie w kodzie

- **Implementacja.** LLM był wykorzystywany do współtworzenia kodu silnika gry (`game.py`: generowanie ruchów, bicia łańcuchowe, lotne damki, promocja), algorytmów MCTS (`mcts.py`: warianty UCT, RAVE, progressive bias, root parallelization), graczy (`players.py`), serwera Flask (`app.py`) oraz skryptów eksperymentalnych (`experiments.py`, `hypothesis_lib.py`, `run_h1.py`–`run_h4.py`, `analyze_results.py`, `plot_results.py`) – na podstawie specyfikacji zasad gry, wybranych wariantów MCTS (sekcja 1.2) i hipotez badawczych (sekcja 2) określonych przez autorów.
- **Testy.** LLM pomagał w pisaniu i utrzymaniu testów jednostkowych (`tests/test_game.py`, `tests/test_players.py`, `tests/test_hypothesis_lib.py`), w tym przypadków granicznych dla reguł gry (bicia łańcuchowe, lotne damki, brak obowiązku bicia) oraz dla pipeline'u `run` → `stats` → `plots`.
- **Debugowanie i refaktoryzacja.** LLM był wykorzystywany do lokalizowania i naprawy błędów (np. w generowaniu ruchów damki, kalibracji budżetu czasowego eksperymentów) oraz do drobnych refaktoryzacji kodu.
- **Co nie pochodzi od LLM.** Wybór algorytmu (MCTS/UCT), zasad wariantu warcabów, modyfikacji do porównania (RAVE, progressive bias) na podstawie przeglądu literatury [2], treść czterech hipotez badawczych (H1–H4) oraz interpretacja i ocena wyników eksperymentów są decyzjami i analizą autorów.

10.2 Wykorzystanie w raporcie

- LLM był wykorzystywany do redakcji i formatowania niniejszego dokumentu $\text{\LaTeX}$ – m.in. uzupełnienia sekcji „Wyniki” dla H1–H4 na podstawie plików `output/h{i}/stats.csv` i `stats.json` wygenerowanych przez skrypty `run_h{i}.py`, wstawienia wykresów (`output/h{i}/plots/*`) oraz sformułowania opisów słownych („Obserwacje”, werdykty H1–H4) na podstawie tych danych liczbowych.
- Treść sekcji 1 (opis problemu, przegląd literatury) i 2 (hipotezy i ich uzasadnienie) została rozwinięta przez LLM na podstawie konspektu projektu (`docs/konspekt/konspekt.tex`), autorstwa zespołu.
- Streszczenie (abstrakt) zostało wygenerowane przez LLM jako synteza wyników poszczególnych hipotez, a następnie zweryfikowane przez autorów pod kątem zgodności z danymi.
- Wszystkie liczby, tabele, wykresy i werdykty (potwierdzona / niepotwierdzona) odpowiadają surowym wynikom eksperymentów zapisanym w katalogu `output/` – LLM nie generował żadnych danych liczbowych „od siebie”.

Literatura

- [1] L. Kocsis and C. Szepesvári, *Bandit based Monte-Carlo Planning*, ECML, 2006.
- [2] M. Świechowski, K. Godlewski, B. Sawicki, J. Mańdziuk, *Monte Carlo Tree Search: a review of recent modifications and applications*, Artificial Intelligence Review, 56:2497–2562, 2023.

- [3] S. Gelly and D. Silver, *Combining online and offline knowledge in UCT*, ICML, 2007.
- [4] S. Gelly and D. Silver, *Monte-Carlo tree search and rapid action value estimation in computer Go*, Artificial Intelligence, 175(11):1856–1875, 2011.
- [5] G. M. J.-B. Chaslot, M. H. M. Winands, H. J. van den Herik, J. W. H. M. Uiterwijk, B. Bouzy, *Progressive strategies for Monte-Carlo tree search*, New Mathematics and Natural Computation, 4(3):343–357, 2008.